時間複雜度 & 暴力

Time Complexity & Brute Force

謝嘉穎

May 24 2026

自我介紹

- Blame
- 2026 全國賽二等獎
- 2026 TOI 選訓營 2!

前言

資讀這屆原本要辦線下，但我有點撞到事情，所以這堂暫改線上

前言

資讀這屆原本要辦線下，但我有點撞到事情，所以這堂暫改線上

上課的時候有問題可以直接提問

Introduction

什麼是時間複雜度

時間複雜度 (Time Complexity)

是一種評估程式可以跑多快的數學表示法

什麼是時間複雜度

時間複雜度 (Time Complexity)

是一種評估程式可以跑多快的數學表示法

在競賽中，程式跑的多快可以決定拿到的分數

漸進符號

要表示複雜度的話，我們需要用到 **漸進符號**來表示

最常見的漸進符號是 O

漸進符號

要表示複雜度的話，我們需要用到 **漸進符號**來表示

最常見的漸進符號是 O

數學的定義是這樣子

定義 (O)

$f(n) = O(g(n)) \Leftrightarrow \exists n_0, c > 0,$ such that $\forall n \geq n_0, f(n) \leq c \cdot g(n)$

漸進符號

要表示複雜度的話，我們需要用到 **漸進符號**來表示

最常見的漸進符號是 O

數學的定義是這樣子

定義 (O)

$f(n) = O(g(n)) \Leftrightarrow \exists n_0, c > 0,$ such that $\forall n \geq n_0, f(n) \leq c \cdot g(n)$

簡單來講是這樣

定義 (O)

$O(g(n))$ 用來代表一個算法的最糟上界量級差不多 $g(n)$

Example

舉個例子

```
for(int i = 1; i <= n; i++) {
    cout << i << '␣';
}
```

這份 code 會跑 n 次

所以時間複雜度是 $O(n)$

Example

```
int cnt = 0;
for(int i = 1; i <= n; i++) {
    int a; cin >> a;
    if(a == 67) {
        break;
    }
    cnt++;
}
cout << cnt << '\n';
```

Example

```
int cnt = 0;
for(int i = 1; i <= n; i++) {
    int a; cin >> a;
    if(a == 67) {
        break;
    }
    cnt++;
}
cout << cnt << '\n';
```

雖然 $a_1 = 67$ 有可能達到 $O(1)$，但是最糟是 $a_i \neq 67$

需要跑 n 次，所以時間複雜度是 $O(n)$

O 表示法

而對於 O 的表示法來講，大概有幾個規則

性質 (忽略常數)

O 裡面只放變數，常數通常省略

也就是說 $O(5N) = O(67N) = O(\frac{1}{67}N) = O(N)$

O 表示法

而對於 O 的表示法來講，大概有幾個規則

性質 (忽略常數)

O 裡面只放變數，常數通常省略

也就是說 $O(5N) = O(67N) = O(\frac{1}{67}N) = O(N)$

性質 (忽略成長較慢項)

對於同一種變數構成的項次，隨著變數增長，成長較慢的項次將被忽略

舉例來說就是，$O(N^2 + N) = O(N^2)$

Example

```
int n, m; cin >> n >> m;
vector<int> A(n), B(m);
for(int i = 0; i < n; i++) cin >> A[i];
for(int j = 0; j < m; j++) cin >> B[j];
```

時間複雜度 $O(n+m)$

Example

```
int mx = 0;
for(int i = 0; i < n; i++) {
    mx = max(mx, A[i]);
}
int mn = 67;
for(int i = n-1; i >= 0; i--) {
    mn = min(mn, A[i]);
}
```

時間複雜度 $O(n)$

Example

```
int n; cin >> n;
vector<int> A(n);
for(int i = 0; i < n; i++) cin >> A[i];
int ans = 0;
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        ans += A[i] * A[j];
    }
}
cout << ans << '\n';
```

時間複雜度 $O(n^2)$

Example

```
int AA_sum = 0, BB_sum = 0;
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        AA_sum += A[i] * A[j];
    }
    for(int k = 0; k < m; k++) {
        BB_sum += A[i] * B[k];
    }
}
```

時間複雜度 $O(n^2 + nm)$

其他漸進符號

定義 (Θ)

$f(n) = \Theta(g(n)) \Leftrightarrow \exists n_0, c_1, c_2 > 0$ such that $\forall n \geq n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

定義 (Ω)

$f(n) = \Omega(g(n)) \leftrightarrow \exists n_0, c > 0$ such that $\forall n \geq n_0, c \cdot g(n) \leq f(n)$

定義 (ω)

$f(n) = \omega(g(n)) \leftrightarrow \exists n_0, c > 0$ such that $\forall n \geq n_0, c \cdot g(n) < f(n)$

Applications

Applications

常見複雜度

級數複雜度

常數

常見複雜度

學會計算複雜度後

我們可以透過觀察題目給的範圍去判斷跑的跑不動

常見複雜度

學會計算複雜度後

我們可以透過觀察題目給的範圍去判斷跑的跑不動

對於現代機器來講，通常 1s 可以跑 5×10^8 左右

常見複雜度

- $N \leq 10^{18}$: $O(\log N)$

常見複雜度

- $N \leq 10^{18}$: $O(\log N)$
- $N \leq 10^{8}$: $O(N)$

常見複雜度

- $N \leq 10^{18}$: $O(\log N)$
- $N \leq 10^{8}$: $O(N)$
- $N \leq 10^{6}$: $O(N \log N)$

常見複雜度

- $N \leq 10^{18}$: $O(\log N)$
- $N \leq 10^{8}$: $O(N)$
- $N \leq 10^{6}$: $O(N \log N)$
- $N \leq 10^{5}$: $O(N\sqrt{N})$

常見複雜度

- $N \leq 10^{18}$: $O(\log N)$
- $N \leq 10^{8}$: $O(N)$
- $N \leq 10^{6}$: $O(N \log N)$
- $N \leq 10^{5}$: $O(N\sqrt{N})$
- $N \leq 10^{3}$: $O(N^2)$

常見複雜度

- $N \leq 10^{18}$: $O(\log N)$
- $N \leq 10^{8}$: $O(N)$
- $N \leq 10^{6}$: $O(N \log N)$
- $N \leq 10^{5}$: $O(N\sqrt{N})$
- $N \leq 10^{3}$: $O(N^2)$
- $N \leq 500$: $O(N^3)$

常見複雜度

- $N \leq 10^{18}$: $O(\log N)$
- $N \leq 10^{8}$: $O(N)$
- $N \leq 10^{6}$: $O(N \log N)$
- $N \leq 10^{5}$: $O(N\sqrt{N})$
- $N \leq 10^{3}$: $O(N^2)$
- $N \leq 500$: $O(N^3)$
- $N \leq 100$: $O(N^4)$

常見複雜度

- $N \leq 10^{18}$: $O(\log N)$
- $N \leq 10^{8}$: $O(N)$
- $N \leq 10^{6}$: $O(N \log N)$
- $N \leq 10^{5}$: $O(N\sqrt{N})$
- $N \leq 10^{3}$: $O(N^2)$
- $N \leq 500$: $O(N^3)$
- $N \leq 100$: $O(N^4)$
- $N \leq 20$: $O(2^N)$

常見複雜度

- $N \leq 10^{18}$: $O(\log N)$
- $N \leq 10^{8}$: $O(N)$
- $N \leq 10^{6}$: $O(N \log N)$
- $N \leq 10^{5}$: $O(N\sqrt{N})$
- $N \leq 10^{3}$: $O(N^2)$
- $N \leq 500$: $O(N^3)$
- $N \leq 100$: $O(N^4)$
- $N \leq 20$: $O(2^N)$
- $N \leq 8$: $O(N!)$

跑的跑不動?

要背上面的表嗎?

跑的跑不動?

要背上面的表嗎?

忘記的話，我比賽現場要怎麼知道會過?

跑的跑不動?

要背上面的表嗎?

忘記的話，我比賽現場要怎麼知道會過?

把數字代進去。

Applications

常見複雜度

級數複雜度

常數

級數複雜度

當然，世界不是那麼美好的

有時候有些時間複雜度有點難分析

級數複雜度

當然，世界不是那麼美好的

有時候有些時間複雜度有點難分析

$O(1) + O(1) + O(2) + \cdots + O(N)$?

級數複雜度

當然，世界不是那麼美好的

有時候有些時間複雜度有點難分析

$O(1) + O(1) + O(2) + \cdots + O(N)$?

等差級數

$$O(\frac{(1+N) \times N}{2}) = O(N^2)$$

級數複雜度

$$O(1) + O(2) + O(4) + \cdots + O(2^N)?$$

級數複雜度

$$O(1) + O(2) + O(4) + \cdots + O(2^N)?$$

等比級數

$$O(\frac{2^{N+1} - 1}{1}) = O(2^N)$$

級數複雜度

$$O(N) + O(\tfrac{1}{2}N) + O(\tfrac{1}{3}N) + \cdots + O(\tfrac{1}{N}N)?$$

級數複雜度

$O(N) + O(\frac{1}{2}N) + O(\frac{1}{3}N) + \cdots + O(\frac{1}{N}N)$?

$O(N \log N)$

級數複雜度

$O(N) + O(\frac{1}{2}N) + O(\frac{1}{3}N) + \cdots + O(\frac{1}{N}N)$?

$O(N \log N)$

$O(N) + O(\frac{1}{2}N) + O(\frac{1}{4}N + \cdots + O(\frac{1}{2^k}N)), 2^k \leq N$?

級數複雜度

$O(N) + O(\frac{1}{2}N) + O(\frac{1}{3}N) + \cdots + O(\frac{1}{N}N)$?

$O(N \log N)$

$O(N) + O(\frac{1}{2}N) + O(\frac{1}{4}N + \cdots + O(\frac{1}{2^k}N)), 2^k \leq N$?

$O(N)$

Applications

常見複雜度

級數複雜度

常數

常數

時間複雜度常常忽略常數

是因為常數往往影響很小，在變數的快增長下顯得不重要

常數

時間複雜度常常忽略常數

是因為常數往往影響很小，在變數的快增長下顯得不重要

但是有時候，常數也可能導致 TLE

像是 $O(N \log N)$ 的複雜度可以過

但是你外面帶個常數 67，就倒了。

練習

比大小

- $O(N)$ v.s. $O(4N)$

比大小

- $O(N)$ v.s. $O(4N)$
- $O(N^2)$ v.s. $O(1000N)$

比大小

- $O(N)$ v.s. $O(4N)$
- $O(N^2)$ v.s. $O(1000N)$
- $O(N \log N)$ v.s. $O(N \log^2 N)$

比大小

- $O(N)$ v.s. $O(4N)$
- $O(N^2)$ v.s. $O(1000N)$
- $O(N \log N)$ v.s. $O(N \log^2 N)$
- $O(\log_{10} N)$ v.s. $O(\log_3 N)$

比大小

- $O(N)$ v.s. $O(4N)$
- $O(N^2)$ v.s. $O(1000N)$
- $O(N \log N)$ v.s. $O(N \log^2 N)$
- $O(\log_{10} N)$ v.s. $O(\log_3 N)$
- $O(N^2 M)$ v.s. $O(N M^2)$

比大小

- $O(N)$ v.s. $O(4N)$
- $O(N^2)$ v.s. $O(1000N)$
- $O(N \log N)$ v.s. $O(N \log^2 N)$
- $O(\log_{10} N)$ v.s. $O(\log_3 N)$
- $O(N^2 M)$ v.s. $O(N M^2)$
- $O(N\sqrt{N})$ v.s. $O(N \log^3 N)$

比大小

- $O(N)$ v.s. $O(4N)$
- $O(N^2)$ v.s. $O(1000N)$
- $O(N \log N)$ v.s. $O(N \log^2 N)$
- $O(\log_{10} N)$ v.s. $O(\log_3 N)$
- $O(N^2 M)$ v.s. $O(N M^2)$
- $O(N\sqrt{N})$ v.s. $O(N \log^3 N)$
- $O(2^N)$ v.s. $O(3^N)$

比大小

- $O(N)$ v.s. $O(4N)$
- $O(N^2)$ v.s. $O(1000N)$
- $O(N \log N)$ v.s. $O(N \log^2 N)$
- $O(\log_{10} N)$ v.s. $O(\log_3 N)$
- $O(N^2 M)$ v.s. $O(N M^2)$
- $O(N\sqrt{N})$ v.s. $O(N \log^3 N)$
- $O(2^N)$ v.s. $O(3^N)$
- $O(N!)$ v.s. $O(2^N)$

Tier List

Brute Force

什麼是暴力?

暴力通常指得是一些複雜度不怎麼好，但可以將答案正確算出來的算法

通常有著

- 模擬
- 枚舉

什麼是暴力?

暴力通常指得是一些複雜度不怎麼好，但可以將答案正確算出來的算法

通常有著

- 模擬
- 枚舉

在 OI 中，比賽越來越難，暴力寫得快寫得穩，往往更佔優勢。

暴力可以幹嘛?

雖然暴力通常無法過題目的時間限制

但可以拿來驗證你的想法 (或對於題目的理解) 是否正確

有時候可以撈一點分數，有時候寫完暴力就可以想到正解怎麼做

模擬

模擬

模擬的精隨在於盲從

題目跟你說要做什麼，你就跟著它做什麼

模擬

模擬的精隨在於盲從

題目跟你說要做什麼，你就跟著它做什麼

有點像是題目是你 ICPC 隊友，他已經跟你講要怎麼做了

你要做的事情就是把這些東西實作出來

題目

習題 (TOJ 1076 打怪遊戲)

n 個人在打 BOSS，BOSS 有 H 的血量

一開始有一個輪流打怪的隊伍順序 $p_1, p_2, \ldots, p_n$

每次都是隊伍的第一個先攻擊，每次花費 a_i 元，消耗掉 BOSS a_i 的血量

假如 BOSS 死掉了 (血量 ≤ 0)，最後一個攻擊他的人贏，離開隊伍

然後 BOSS 復活 (擁有 H 的血量)，剩下的人繼續攻擊，直到所有人都離開隊伍

求每個人離開隊伍的先後順序，以及總花費金額

$1 \leq n, h, a_i \leq 2000$

枚舉

例題一

例題 (ABC 214 B - How many?)

給 S T

求有多少個非負整數三元組 (a, b, c) 滿足 $a + b + c \leq S$ 和 $a \times b \times c \leq T$

$S \leq 100$

$T \leq 10^4$

例題一解答

對於枚舉來說，值域的大小與枚舉的複雜度有關

那我們不妨分析一下值域大小

例題一解答

對於枚舉來說，值域的大小與枚舉的複雜度有關

那我們不妨分析一下值域大小

假如我們依照 $a \times b \times c \leq T$ 枚舉，然後檢查 $a + b + c$ 是否 $\leq S$

例題一解答

對於枚舉來說，值域的大小與枚舉的複雜度有關

那我們不妨分析一下值域大小

假如我們依照 $a \times b \times c \leq T$ 枚舉，然後檢查 $a + b + c$ 是否 $\leq S$

先不考慮 $a, b, c = 0$ 的情況

$a = i$ 的時候，b 會枚舉 1 到 $\lfloor \frac{T}{i} \rfloor$

$a = i, b = j$ 時，c 會枚舉 1 到 $\lfloor \frac{T}{i \times j} \rfloor$

也就是說，總共會枚舉 $\sum_{i=1}^{T} \sum_{j=1}^{\lfloor \frac{T}{i} \rfloor} \lfloor \frac{T}{i \times j} \rfloor$ 次

例題一解答

對著剛剛的式子亂算，大概會得到 $O(T \log^2 T)$ 的複雜度

這樣感覺好複雜，而且還要判零的 case

例題一解答

對著剛剛的式子亂算，大概會得到 $O(T\log^2 T)$ 的複雜度

這樣感覺好複雜，而且還要判零的 case

我們換個枚舉方式，對著 $a+b+c \le S$ 枚舉，檢查 $a \times b \times c \le T$

$a = i$，b 從 $0 \sim S - i$

$a = i, b = j$，c 從 $0 \sim S - i - j$

總枚舉次數 $\sum_{i=0}^{S} \sum_{j=0}^{S-i} (S - i - j + 1)$

例題一解答

對著剛剛的式子亂算，大概會得到 $O(T\log^2 T)$ 的複雜度

這樣感覺好複雜，而且還要判零的 case

我們換個枚舉方式，對著 $a+b+c \le S$ 枚舉，檢查 $a \times b \times c \le T$

$a = i$，b 從 $0 \sim S - i$

$a = i, b = j$，c 從 $0 \sim S - i - j$

總枚舉次數 $\sum_{i=0}^{S} \sum_{j=0}^{S-i} (S - i - j + 1)$

複雜度差不多 $O(\binom{S+3}{3}) = O(\frac{S^3}{6})$

值域差別

發現剛剛有兩種枚舉方式

且複雜度看起來差別很大，$O(T\log^2 T)$ $O(\frac{S^3}{6})$

值域差別

發現剛剛有兩種枚舉方式

且複雜度看起來差別很大，$O(T\log^2 T)$ $O(\frac{S^3}{6})$

也就是說當 T 變大的時候，對著 S 枚舉會更好

當 S 變大的時候，對著 T 枚舉會更好

題目

習題 (TOJ 1102 Brain Rot 數字 (Easy Version))

求 $1 \sim N$ 之中，有多少個數字的位數和是 67 的倍數，且本身也是 67 的倍數

$1 \leq N \leq 10^9$

習題 (TOJ 1111 闇影沒有睿智)

給三數 A B C

令數列 D 為滿足 $([D_i \equiv 0 \pmod{A}] + [D_i \equiv 0 \pmod{B}] + [D_i \equiv 0 \pmod{C}] \geq 2)$ 之數字組成，由小到大

求 D_k

$1 \leq A, B, C \leq 1000$

$1 \leq k \leq 3 \times 10^6$

Applications

Applications

全排列

枚舉子集合

BFS 枚舉

例題一

例題 (TOJ 842 奇妙的排序)

給一數 N

問有多少個質數可以透過將 N 的某些位數拿出來，任意排序後得到

$1 \leq N \leq 10^6$

例題一解答

暴力把所有 N 可以透過交換位數的數字枚舉出來

那要怎麼枚舉?

例題一解答

暴力把所有 N 可以透過交換位數的數字枚舉出來

那要怎麼枚舉?

```
next_permutation(a.begin(), a.end());
```

一個 C++ 的語法，他會把 vector a 變成**下一個字典序的排列**，也會回傳有沒有這個排列

Code

```
string s; cin >> s;
sort(s.begin(), s.end());
do {
    int n = stoi(s);
    // do something
} while(next_permutation(s.begin(), s.end()));
```

時間複雜度為 $O(n!)$

Applications

全排列

枚舉子集合

BFS 枚舉

例題二

例題 (TOJ 656 枚舉子集和)

有一個長度為 n 的陣列 A

計算有多少個子集合的和為 k

$1 \leq n \leq 20$

例題二解答

集合 $S = \{A_1, A_2, A_3, \ldots, A_n\}$

要求的就是 $\sum_{T \subset S} [\sum_{a \in T} a = k]$

例題二解答

集合 $S = \{A_1, A_2, A_3, \ldots, A_n\}$

要求的就是 $\sum_{T \subset S} [\sum_{a \in T} a = k]$

假如你有算過排列組合，那應該知道總共有 $2^{|S|}$ 個子集合

為什麼呢?

例題二解答

集合 $S = \{A_1, A_2, A_3, \ldots, A_n\}$

要求的就是 $\sum_{T \subset S} [\sum_{a \in T} a = k]$

假如你有算過排列組合，那應該知道總共有 $2^{|S|}$ 個子集合

為什麼呢?

每個元素有跟沒有，兩種狀態，總共 $|S|$ 個

例題二解答

枚舉子集合的方式也是一樣

遞迴下去，每次遇到一個元素，就分岔出去兩種情況

```
void dfs(int i, long long sum = 0) {
    if(i == n) { // 0 ~ n-1, n is the end
        if(sum == k) ans++;
        return;
    }
    dfs(i+1, sum); // 沒有
    dfs(i+1, sum+A[i]); // 有 Ai
}
```

時間複雜度為 $O(2^n)$

位元枚舉

枚舉方式也可以用位元枚舉

我們用數字 i 代表著集合 T 的狀態

假如數字 i 的第 j 位位元是 1 就代表著集合 T 有 A_j

```
for(int i = 0; i < (1<<n); i++) {
    int sum = 0;
    for(int j = 0; j < n; j++) if(i&(1<<j)){
        sum += A[j];
    }
    if(sum == k) ans++;
}
```

時間複雜度 $O(2^n \cdot n)$，但常數很好

題目

習題 (CSES 1623 Apple Division)

有 n 個蘋果，每個有各自的重量 p_i

將蘋果分成兩堆，使得兩堆的重量總和相差最小

求相差最小可以是多少

$1 \leq n \leq 20$

$1 \leq p_i \leq 10^9$

Applications

全排列

枚舉子集合

BFS 枚舉

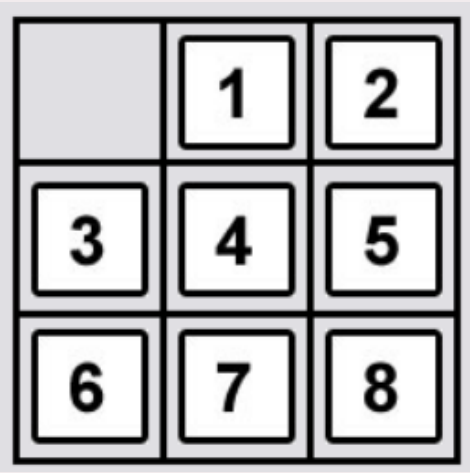
例題三

例題 (TOJ 429 D. 巨人之杯)

有 3×3 的拼圖，$1 \sim 8$ 然後少一格

可以讓在空格四周的數字移動到空格上

目標是讓他變成



T 筆測資，輸出最少步數，假如無法達到，輸出 QQ

$1 \leq T \leq 20$

例題三解答

發現操作是將數字移動到空格上，發現這個操作是可逆的

也就是說開始狀態 $\rightarrow$ 到達狀態等於到達狀態 $\rightarrow$ 開始狀態

例題三解答

發現操作是將數字移動到空格上，發現這個操作是可逆的

也就是說開始狀態 $\rightarrow$ 到達狀態等於到達狀態 $\rightarrow$ 開始狀態

考慮直接將開始狀態能到達的狀態枚舉出來，並記錄最短操作次數

這樣就可以直接查表詢問了!

那要怎麼暴力找所有狀態呢?

例題三解答

發現操作是將數字移動到空格上，發現這個操作是可逆的

也就是說開始狀態 $\rightarrow$ 到達狀態等於到達狀態 $\rightarrow$ 開始狀態

考慮直接將開始狀態能到達的狀態枚舉出來，並記錄最短操作次數

這樣就可以直接查表詢問了!

那要怎麼暴力找所有狀態呢?

以後會介紹一個演算法叫做 BFS，可以做到剛剛所說的事情

優化

優化

剪枝

Meet in the Middle

例題一

例題 (TOJ 657 八皇后問題)

8×8 的棋盤上要放八隻皇后，有些位置不能放皇后

問有多少種放法可以放下八隻皇后，並且彼此攻擊不到彼此

例題一解答

發現總共有 $64 \times (64-1) \times \cdots \times (64-7)$ 種放置方法

這樣太多了，有更好的枚舉方法嗎?

例題一解答

發現總共有 $64 \times (64-1) \times \cdots \times (64-7)$ 種放置方法

這樣太多了，有更好的枚舉方法嗎?

我們發現有些放法其實根本不可能放得下

對於兩隻皇后來講，只要他們所在行跟所在列相同，那一定是不合法的放置方法

所以我們每次都枚舉每一列的皇后要放在哪一行，並且第 i 列的皇后不能與 $1 \sim i-1$ 列的皇后同一行

這樣複雜度就被壓到 $O(8! \times$ 檢查斜線時間$)$

題目

習題 (CSES 1625 Grid Path Description)

有一個 7×7 的格子，要從左上走到左下，且將所有格子都走過 (不重複走)

那麼路徑長度一定是 48

今天給你長度為 48 的字串，包含 (L, R, U, D, ?)

你可以將? 視為任意方向，問有多少個走法符合字串的走法，且可以從左上走到左下，並且將所有格子都走過

優化

剪枝

Meet in the Middle

例題二

例題 (CSES 1628 Meet in the Middle)

有 n 個元素 t_i

問有多少個子集合總和是 x

$1 \leq n \leq 40$

例題二解答

枚舉子集合的做法，$O(2^n) = O(2^{40})$，爆掉了

有沒有更快的做法?

例題二解答

枚舉子集合的做法，$O(2^n) = O(2^{40})$，爆掉了

有沒有更快的做法?

發現枚舉前 $1 \sim \lfloor \frac{n}{2} \rfloor$ 個子集合

將每個子集合的總和記錄下來

再枚舉 $\lfloor \frac{n}{2} \rfloor + 1 \sim n$ 的子集合

這樣右邊子集合的總和 a 可以用二分搜找到左邊的子集合是否存在總合為 $k - a$ 的

例題二解答

枚舉子集合的做法，$O(2^n) = O(2^{40})$，爆掉了

有沒有更快的做法?

發現枚舉前 $1 \sim \lfloor \frac{n}{2} \rfloor$ 個子集合

將每個子集合的總和記錄下來

再枚舉 $\lfloor \frac{n}{2} \rfloor + 1 \sim n$ 的子集合

這樣右邊子集合的總和 a 可以用二分搜找到左邊的子集合是否存在總合為 $k - a$ 的

時間複雜度降到 $2^{\lfloor \frac{n}{2} \rfloor} \times \log(\lfloor \frac{n}{2} \rfloor)$

End